

Lecture 4: NFS

(Distrib. State + Evolution of v2-v4)

CS 739: Distributed Systems

Prof. Andrea C. Arpaci-Dusseau
University of Wisconsin — Madison
Spring 2025

Today's Topic: NFS

Focus on parts of NFS with State → Design Implications for scalability, semantics, and recovery

1.State: [The Role of Distributed State](#) [Download The Role of Distributed State](#)(skip later sections on Sprite)

John Ousterhout, 1991

2.NFSv2 Background: [Design and Implementation of the Sun Network Filesystem](#) (Focus on material before Implementation)

Sandberg, R., Goldberg, D., Kleiman, S., Walsh, D., and Lyon, B., Proceedings of the Summer 1985 USENIX Conference, Portland OR, June 1985, pp. 119-130.

3.NFSv3: [NFS Version 3: Design and Implementation](#)[Links to an external site.](#) (Focus on material before Section 5: Performance)

Brian Pawlowski and Chet Juszczak and Peter Staubach and Carl Smith and Diane Lebel and Dave Hitz, In USENIX Summer 1994

4.NFSv4: [The NFS Version 4 Protocol](#) [Links to an external site.](#)(Focus on material before Section 10: Attributes)

Brian Pawlowski and Spencer Shepler and Carl Beame and Brent Callaghan and Michael Eisler and David Noveck and David Robinson and Robert Thurlow, Proceedings of the 2nd international system administration and networking conference (SANE2000)

Why Today's Papers? Evolution of NFS protocol adding state to servers (continue stateful servers with AFS)

- Concepts shown in Practice: Many neat details about real systems and practical issues
 - Not SOSP/OSDI/Test of Time winner papers – more implementation details
- Stateless Servers
 - Server does not track which clients have files open()
 - Protocol must contain all info needed to complete requests
- Pros and Cons of Stateless Protocols
 - + Handles crashes easily
 - Confusing semantics for sharing with client caches
 - Poor performance (flush writes to persistent storage + overhead of getattr requests)
- Techniques for Stateful Servers:
 - Leases

Who is John Ousterhout?

Faculty member at UC Berkeley and Stanford University

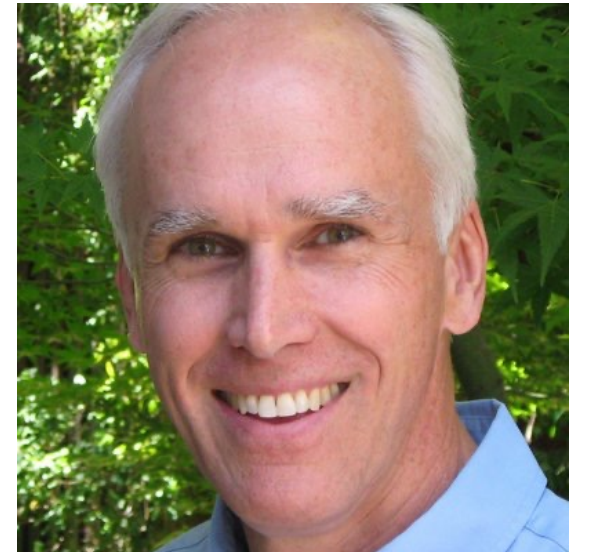
Tcl/Tk

Sprite Operating System

LFS (Log Structured File System)

Raft protocol

Very clear writer!



State and Distributed State

State?

Distributed State?

Benefits of Distributed State?

1)

2)

3)

Challenges of Distributed State?

1)

2)

3)

4)

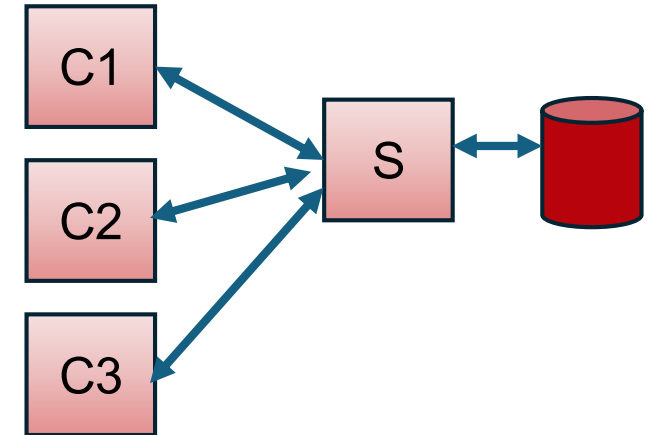
NFSv2 Protocol (vs Implementation)

NFSv2 **servers** are **stateless**?

Can servers keep anything in RAM?

Required property for NFSv2 requests?

What state must clients maintain?

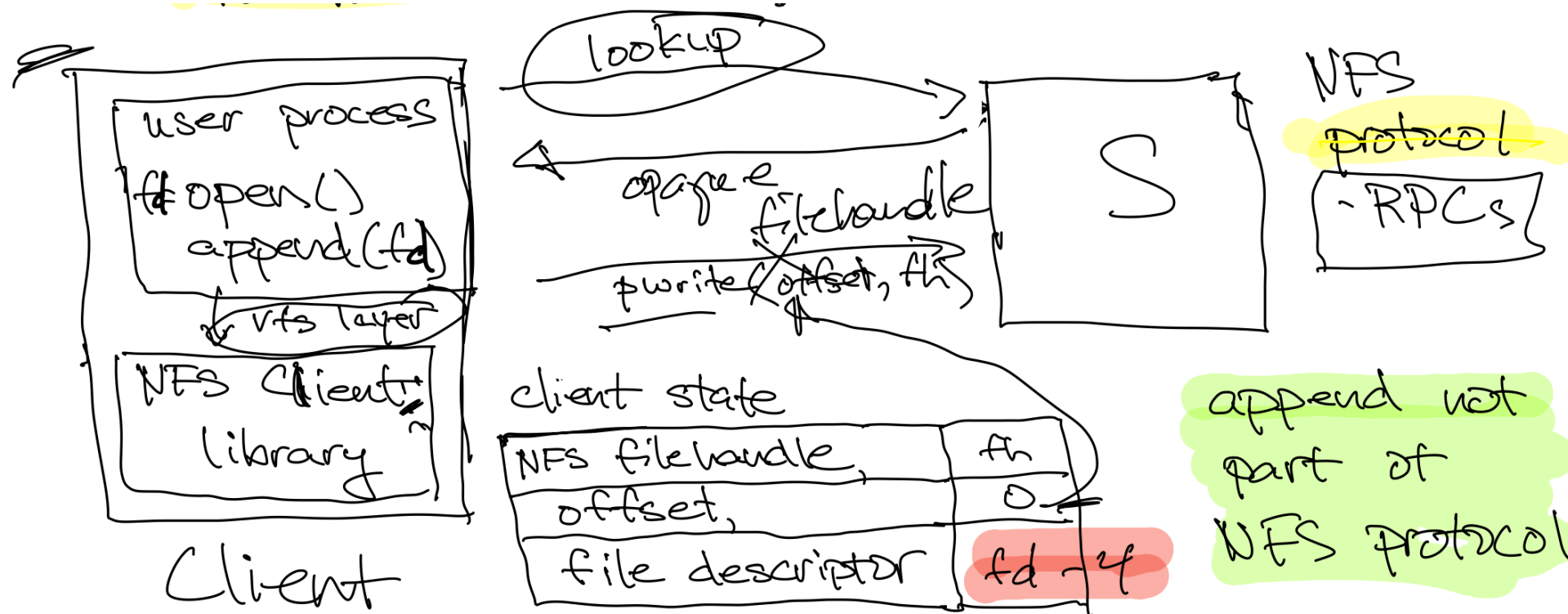


Idempotent Operations

Idempotent:

Can be executed multiple times with same results as only executed once

How can clients (user processes) perform `append()` to NFSv2 file?



Are all NFSv2 ops idempotent?

Example of non-idempotent op?

How did v2 handle non-idempotent ops?

What if server crashes?

Stateless NFSv2 Server: Advantages

Advantages of stateless NFSv2 Server?

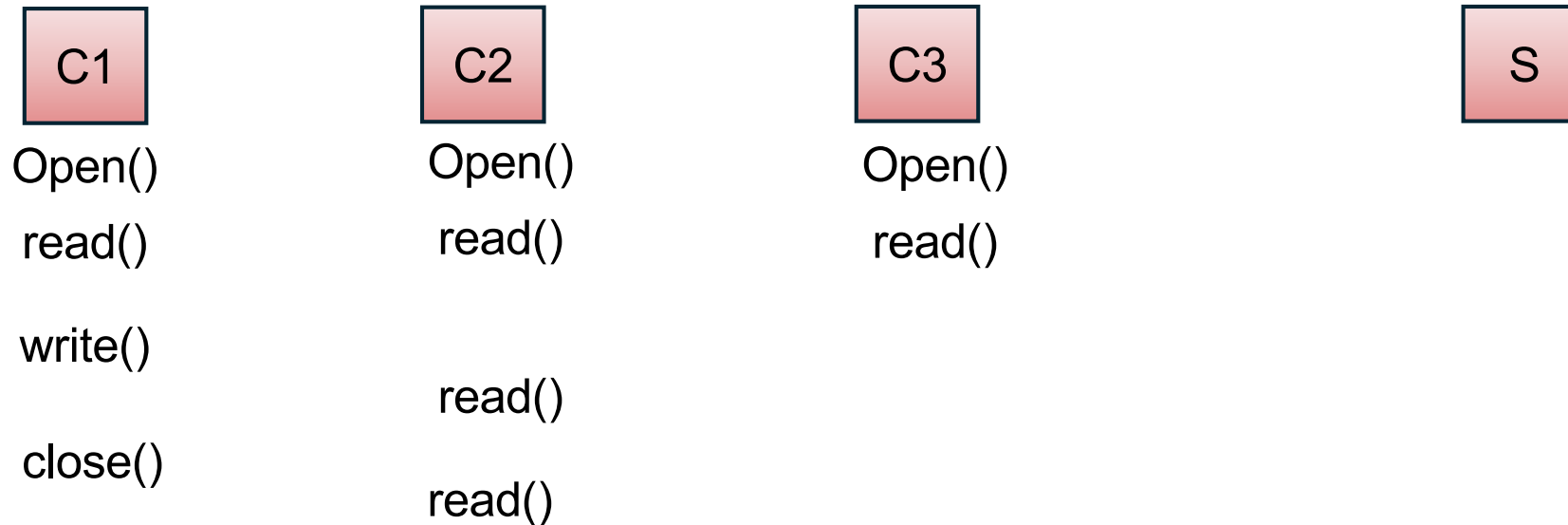
What does client do if server crashes?

What does server do if client crashes?

Stateless NFSv2 Server: Disadvantages

Why do stateless v2 servers hurt performance of writes?

NFSv2 Client Caching: Consistency?



Client Block Cache: Needed for performance! Fundamental problem?

How can clients discover data is stale?

Performance problems?

Rules for checking for changes before read()? next open()?

Distributed Locks

Can stateless server provide locks for files?

Solution?

NFSv3: Addresses 3 problems

1)

2)

3)

NFSv3 Stateless Servers? **Why?**

NFSv3: Improves Write Performance

Asynchronous writes

What must client do if server crashes?

How will client know that server crashed?

NFSv3: Uncommitted Data can be Lost

Can clients read old data from v3 server?

v2 suffered from too many `getattr()` calls; how to reduce?

NFSv3: When are getattr results useless?

Client wants to cache read data for large file; updates some block...

Read: Use cache version or not?

1)

2)

Solution?

NFSv4: Stateful Servers

What is a COMPOUND procedure? Why introduced?

Semantics?

Complexities?

NFSv4: Introduces open() + close()

Why?

When can future open()'s be kept local on client?

NFSv4: Delegation of Open()

What is exclusive delegation? How does it work?

How to discover no longer have exclusive delegation?

1)

2)

What are advantages of delegation?

NFSv4: Crashes with Delegation

Assume **client c1** with ex. delegation crashes;
How does server delegate to different client, c3?

Assume **server** crashes when c1 had delegation;
How does server delegate to c3? What state when server reboots?

NFSv4: Consistency

What is guaranteed if two clients have file open for read/write?

Share reservation:

Client can open file and deny other clients open access to same file

NFSv4: How to detect crashes?

Server must know if **client** crashed

Server tracks:

Client ID (chosen by server), Verifier ID (chosen by client), ...state (locks)...

Client reboots:

New Verifier ID (based on timestamp)

What if **server** crashes?

Summary

NFS v2 + v3: Known for stateless servers

- Clients still need state (track open files); Servers can still cache data for performance
- Open(), close() not part of NFS protocol
- Use polling of getattr() to discover if file has changed
- Poor performance
- Poor consistency

NFSv3: Practical optimizations to improve performance

- Asynchronous writes with commits
- Improved getattr

NFSv4: State on server

- Tracks clients with file open
- Delegation: Keep ops local
 - Mostly performance win
 - Reverts to getattr() if sharing
 - Different than AFS...
- Leases help handle crashes

Announcements

Reading Group 1: Continue for Week 2

- Same group as Week 1

Mini-Project 1: RPC for Client/Server KV Store – Due Wed Feb 19th

- By Wednesday: 1 Partner – Join same “Project 1” group; send email if want to be matched
- Sign up for Cloudlab account
- TA Office Hours: Guanzhou
- Grade with quick demo with us

Wednesday: AFS and Coda

•**AFS Background:** Scale and Performance in a Distributed File System

Howard, J.H., Kazar, M.L., Menees, S.G., Nichols, D.A., Satyanarayanan, M., Sidebotham, R.N., and West, M.J.
ACM Transactions on Computer Systems, Vol. 6, No. 1, February 1988, pp. 51-81.

SIGOPS Hall of Fame Award

•**Coda:** Disconnected Operation in the Coda File System[Links to an external site.](#)

James J. Kistler, M. Satyanarayanan, 13th Symposium on Operating Systems Principles, Asilomar, California, pp. 213-225. October 1991. **SIGOPS Hall of Fame Award**